Assignment

An assignment for class **MET CS 566** at Boston University, instructed by **Ming Zhang**. Student Zhang, Jin Yu submitted on **September 19, 2022**.

Question 1

Python implementation:

```
import numpy as number
import matplotlib.pyplot as plot
import matplotlib.colors as colors

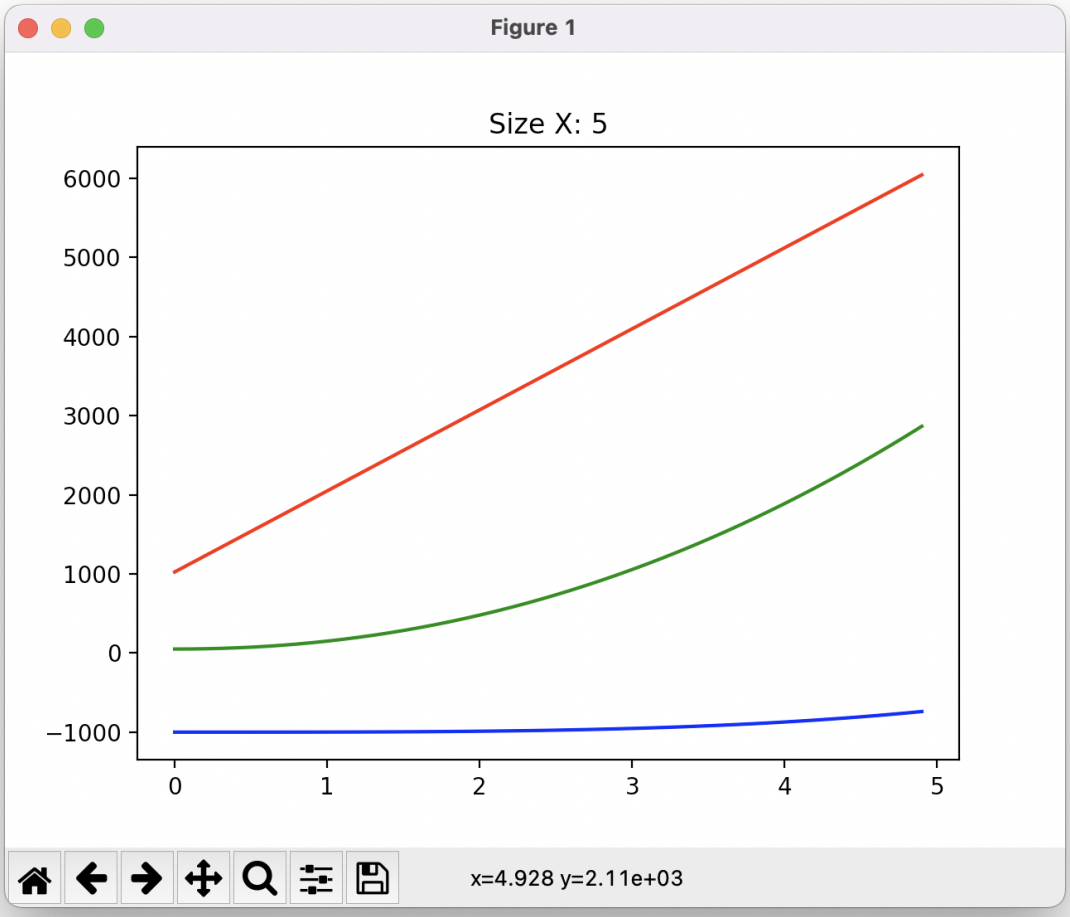
def generate_plot(size_x: int) -> None:
    scale_x = number.arange(0, size_x, 0.1)
    list_plot = [
        {
            'scale_y': (pow(2, 10) * scale_x) + pow(2, 10),
            'color': colors.BASE_COLORS['r']
        },
        {
            'scale_y': pow(scale_x, 3.5) - 1000,
            'color': colors.BASE_COLORS['b']
        },
        {
            'scale_y': (100 * pow(scale_x, 2.1)) + 50,
            'color': colors.BASE_COLORS['g']
        }
    ]
    for item in list_plot:
        plot.plot(scale_x, item['scale_y'], color=item['color'])

    plot.title(f'Size X: {size_x}')
    plot.show();

for x in [5, 15, 50]:
    generate_plot(x)
```

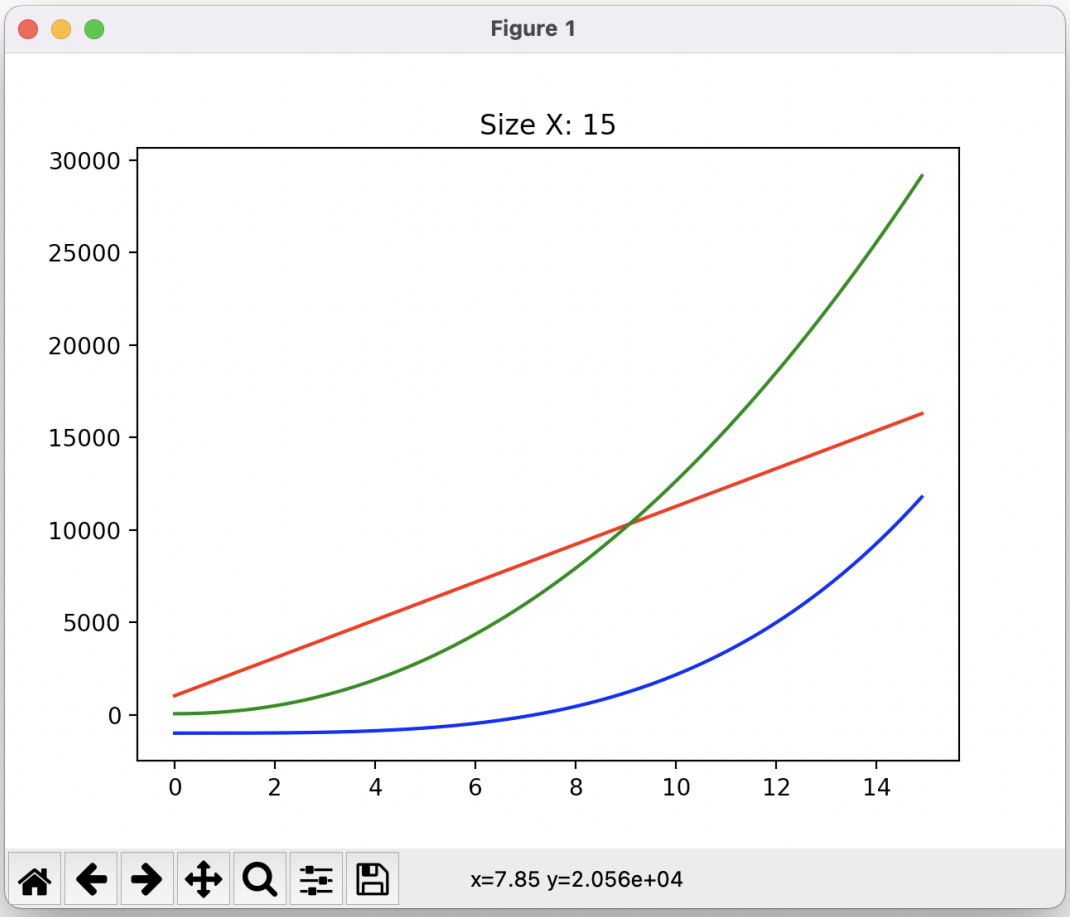
Plot with X-Axis Limited to 5

We can see that $f_1(n)$ increase mostly since it adds 2^{10}, which is 1024 and can dramatically impact the performance with small input. $f_2(n)$ and $f_3(n)$ increase slower since they have smaller or negative lower-order numbers:



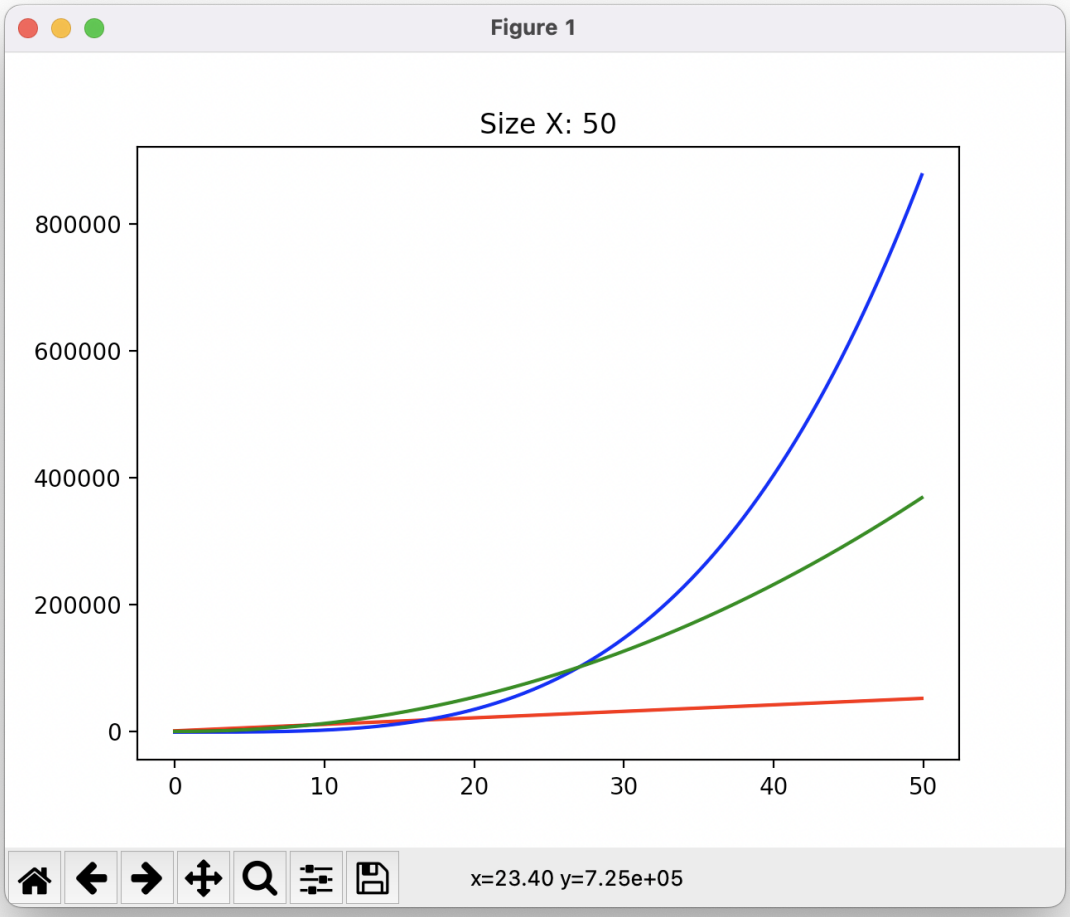
Plot with X-Axis Limited to 15

$f_3(n)$ is now has worst performance and it is mainly due to the function has $100 \times n^{2.1}$ coefficient. $f_1(n)$ and $f_2(n)$ apparently have better performance with input 15:



Plot with X-Axis Limited to 50

The input is large enough to see the impact of the highest-order term. $f_2(n)$ is the worst due to $n^{3.5}$. The second worst is $f_3(n)$ because it has $n^{2.1}$, which is more than **quadratic**. $f_1(n)$ has best performance for large input since it is **linear**:



Question 2

Judge asymptotic notation.

Is $2^{n+1.3} = \mathcal{O}(2^n)$?

Yes. Mathematical proof:

$$f(n) = 2^{n+1.3} = \mathcal{O}(2^n)$$

$$f(n) = 2^{1.3} \times 2^n = \mathcal{O}(2^n)$$

$f(n) = 2^{n+1.3}$. $f(n)$ is said to be $\mathcal{O}(2^n) = \mathcal{O}(g(n))$ if there exist constants c and n_0 such that if $n \geq n_0$ then $f(n) \leq c \times g(n)$. Let's say $c = 3$ and $n_0 = 1$, we have:

$$f(n) \leq c \times g(n)$$

$$f(n) \leq 3 \times g(n)$$

$$2^{1.3} \times 2^1 \leq 3 \times 2^1$$

For $c = 3$, $3 \times 2^n \geq f(n)$ for all $n \geq 1$. Therefore, $2^{n+1.3} = \mathcal{O}(2^n)$.

Is $3^{2 \times n} = \mathcal{O}(3^n)$?

No. Mathematical proof:

$$f(n) = 3^{2 \times n} = \mathcal{O}(3^n)$$

$f(n) = 3^{2 \times n}$. $f(n)$ is said to be $\mathcal{O}(3^n) = \mathcal{O}(g(n))$ if there exist constants c and n_0 such that if $n \geq n_0$ then $f(n) \leq c \times g(n)$. Let's say $c = 3$ and $n_0 = 1$, we have:

$$f(n) \leq c \times g(n)$$

$$f(n) \leq 3 \times g(n)$$

$$3^{2 \times 1} \leq 3 \times 3^1$$

$3^{2 \times 1}$ is less than or equal to 3×3^1. However, when $n_0 > 1$ it is not the case. Let's have $n_0 = 2$:

$$3^{2 \times 2} > 3 \times 3^2$$

3^4 is greater than 3^3. Let's prove by induction with $3^{2 \times n} > 3 \times 3^n$ for all $n > 1$:

$$3^{2 \times (n+1)} > 3 \times 3^{n+1}$$

$$3^{2n+2} > 3^{n+2}$$

$$3^{2n} \times 3^2 > 3^n \times 3^2$$

$$3^{2n} > 3^n$$

Therefore, $3^{2 \times n} \neq \mathcal{O}(3^n)$.

Question 3

For each function $f(n)$ and $g(n)$, check if $(fn) = \theta(g(n))$.

$f(n) = (4 \times n)^{150} + (2 \times n + 1024)^{400} = g(n) - 20 \times n^{400} + (n + 1024)^{200}$

Yes. First we throw away any lower-order terms:

$$f(n) = (2 \times n + 1024)^{400} = g(n) = 20 \times n^{400}$$

Then ignore the leading coefficient:

$$f(n) = n^{400} = g(n) = n^{400}$$

$f(n) = n^{1.4} \times 4^n = g(n) = n^{200} \times 3.99^n$

No. We throw away any lower-order terms:

$$f(n) = 4^n \neq g(n) = n^{200}$$

$f(n) = 2^{log(n)} = g(n) = n^{1024}$

No. First of all, we know $2^{log(n)} < 2^n$. We can find out $2^n < n^{1024}$ then:

$$f(n) = 2^{log(n)} \neq g(n) = n^{1024}$$

Question 4

Analyze the Algorithm 1 and give a tight θ bound on the running time as a function of n.

Algorithm 1 What is the tight Θ of this pseudocode?

```
1: i = 1
2: while i ≤ n do
3:     A[i] = i
4:     i = i + 1
5: end while
6: for j ← 1 to n do
7:     i = j
8:     while i ≤ n do
9:         A[i] = i
10:        i = i + j
11:    end while
12: end for
```

$\theta(n^2)$. It is a **quadratic** function of n. Cost calculation:

Cost of the algorithm line	Number of times the line happens
c_1	1
c_2	$n+1$
c_3	n
c_4	n
c_6	$n+1$
c_7	n
c_8	$\sum_{j=1}^{n}(t_j+1)$
c_9	$\sum_{j=1}^{n} t_j$
c_10	$\sum_{j=1}^{n} t_j$

The whole cost will be:

$$f(n) = c_1 + c_2 \times (n+1) + c_3 \times n + c_4 \times n + c_6 \times (n+1) + c_7 \times n + c_8 \times \sum_{j=1}^{n}(t_j+1) + c_9 \times \sum_{j=1}^{n} t_j + c_{10} \times \sum_{j=1}^{n} t_j$$

Then we can find $f(n)$:

$$\sum_{j=1}^{n}(t_j) = \frac{n(n+1)}{2}, \sum_{j=1}^{n}(t_j+1) = \frac{n(n+1)}{2} + 1$$

$$f(n) = c_1 + n \times (c_3 + c_4 + c_7) + (n+1) \times (c_2 + c_6) + c_8 \times (\frac{n(n+1)}{2} + 1) + (c_9 + c_{10}) \times \frac{n(n+1)}{2}$$

$$f(n) = \frac{(c_8 + c_9 + c_{10})}{2} \times n^2 + (c_1 + c_2 + c_3 + c_4 + c_6 + c_7 + \frac{c_8}{2} + \frac{c_9}{2} + \frac{c_{10}}{2}) \times n + (c_2 + c_6 + c_8)$$

We can express this running time as $a \times n^2 + b \times n + c$ where $a = \frac{(c_8 + c_9 + c_{10})}{2}, b = (c_1 + c_2 + c_3 + c_4 + c_6 + c_7 + \frac{c_8}{2} + \frac{c_9}{2} + \frac{c_{10}}{2}), c = (c_2 + c_6 + c_8)$. It is a **quadratic** function of n.

$f(n) = \theta(g(n))$ if there exists positive numbers n_0, c_1, c_2 such that at and to the right of n_0, the value of $f(n)$ always lies between $c_1 \times g(n)$ and $c_2 \times g(n)$ inclusive.

$$0 \leq c_1 \times g(n) \leq f(n) \leq c_2 \times g(n) \; \forall n \leq n_0$$

Let's claim the following:

$$f(n) = a \times n^2 + b \times n + c = \theta(n^2)$$

Then use the definition of θ to find n_0, c_1, c_2:

$$c_1 \times n^2 \leq a \times n^2 + b \times n + c \leq c_2 \times n^2$$
$$c_1 \leq a + \frac{b}{n} + \frac{c}{n^2} \leq c_2$$

n_0, c_1, c_2:

$$\forall n \geq 1, c_2 \geq a + b + c$$
$$\forall n \geq 1, c_1 \leq a + b + c$$
$$n_0 = 1$$

Other choices of n_0, c_1, c_2 are possible. Since a, b, c are not determined (the cost of the algorithm line) the proof seems redundant and we could just simply throw away all lower-order terms and ignore the leading coefficient to find $\theta(n^2)$.

Question 5

Analyze the Algorithm 2. What is the Big $\mathcal{O}$ on the running time as a function of n.

Algorithm 2 What is the Big O of this pseudocode?

```
1: x = 0
2: for i ← 0 to n do
3:     for j ← 0 to (i × n) do
4:         x = x + 10
5:     end for
6: end for
```

$\mathcal{O}(n^4)$. It is a **quartic** function of n. Cost calculation:

Cost of the algorithm line	Number of times the line happens
c_1	1
c_2	$n+2$
c_3	$\sum_{j=1}^{n^2}(t_j+1)$
c_4	$\sum_{j=1}^{n^2} t_j$

The whole cost will be:

$$f(n) = c_1 + c_2 \times (n+2) + c_3 \times \sum_{j=1}^{n^2}(t_j+1) + c_4 \times \sum_{j=1}^{n^2} t_j$$

Then we can find $f(n)$:

$$\sum_{j=1}^{n^2}(t_j) = \frac{n^2(n^2+1)}{2}, \sum_{j=1}^{n^2}(t_j+1) = \frac{n^2(n^2+1)}{2} + 1$$

$$f(n) = c_1 + c_2 \times n + c_2 \times 2 + (\frac{c_3}{2} + \frac{c_4}{2}) \times n^4 + (\frac{c_3}{2} + \frac{c_4}{2}) \times n^2 + c_3$$

$$f(n) = (\frac{c_3}{2} + \frac{c_4}{2}) \times n^4 + (\frac{c_3}{2} + \frac{c_4}{2}) \times n^2 + c_2 \times n + (c_1 + c_2 \times 2 + c_3)$$

We can express this running time as $a \times n^4 + b \times n^2 + c \times n + d$ where $a = \frac{c_3}{2} + \frac{c_4}{2}, b = (\frac{c_3}{2} + \frac{c_4}{2}), c = c_2, d = (c_1 + c_2 \times 2 + c_3)$. It is a **quartic** function of n.

Let's claim that the function is upper bounded by a function $g(n) = n^4, f(n) = \mathcal{O}(n^4)$, we need to prove $f(n) \leq c_c \times g(n)\ \forall n \geq n_0$. We consider $n_0 = 1$ and c_c is the absolute value of all coefficients:

$$f(n) \leq c_c \times n^4\ \forall n \geq 1$$

$$a \times n^4 + b \times n^2 + c \times n + d \leq c \times (a \times n^4 + b \times n^2 + c \times n + d)^4\ \forall n \geq 1$$

Since a, b, c, d and c_c are all positive numbers, we have prove that $f(n)$ is always smaller or equal to a constant c_c multiply to $(a \times n^4 + b \times n^2 + c \times n + d)^4$.